ERBD 2026

Desenvolvimento de Agentes de IA e Integração com Bases de Dados Heterogêneas

Prof. Dr. Marlon Marcon
UTFPR - campus Dois vizinhos

UTFPR UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ CÂMPUS DOIS VIZINHOS
SBC Sociedade Brasileira de Computação
Cebd SBC
XXI ERBD 2026 DOIS VIZINHOS - PR de 22 a 24/04

CONTEÚDO DO MÓDULO

O que vamos explorar

01 Contextualização Histórica

Da rigidez dos Data Warehouses e ETLs à flexibilidade dos Data Lakes e arquiteturas modernas como Data Mesh e Data Fabric

02 O Desafio Atual

Silos de informação e heterogeneidade: o cenário fragmentado das organizações modernas e seus impactos na tomada de decisão

03 A Ascensão dos Agentes de IA

Uma solução transformadora: como os agentes de IA modernos utilizam LLMs para democratizar o acesso aos dados empresariais

04 Model Context Protocol

Fundamentos das arquitetura do MCP

01

Contextualização Histórica

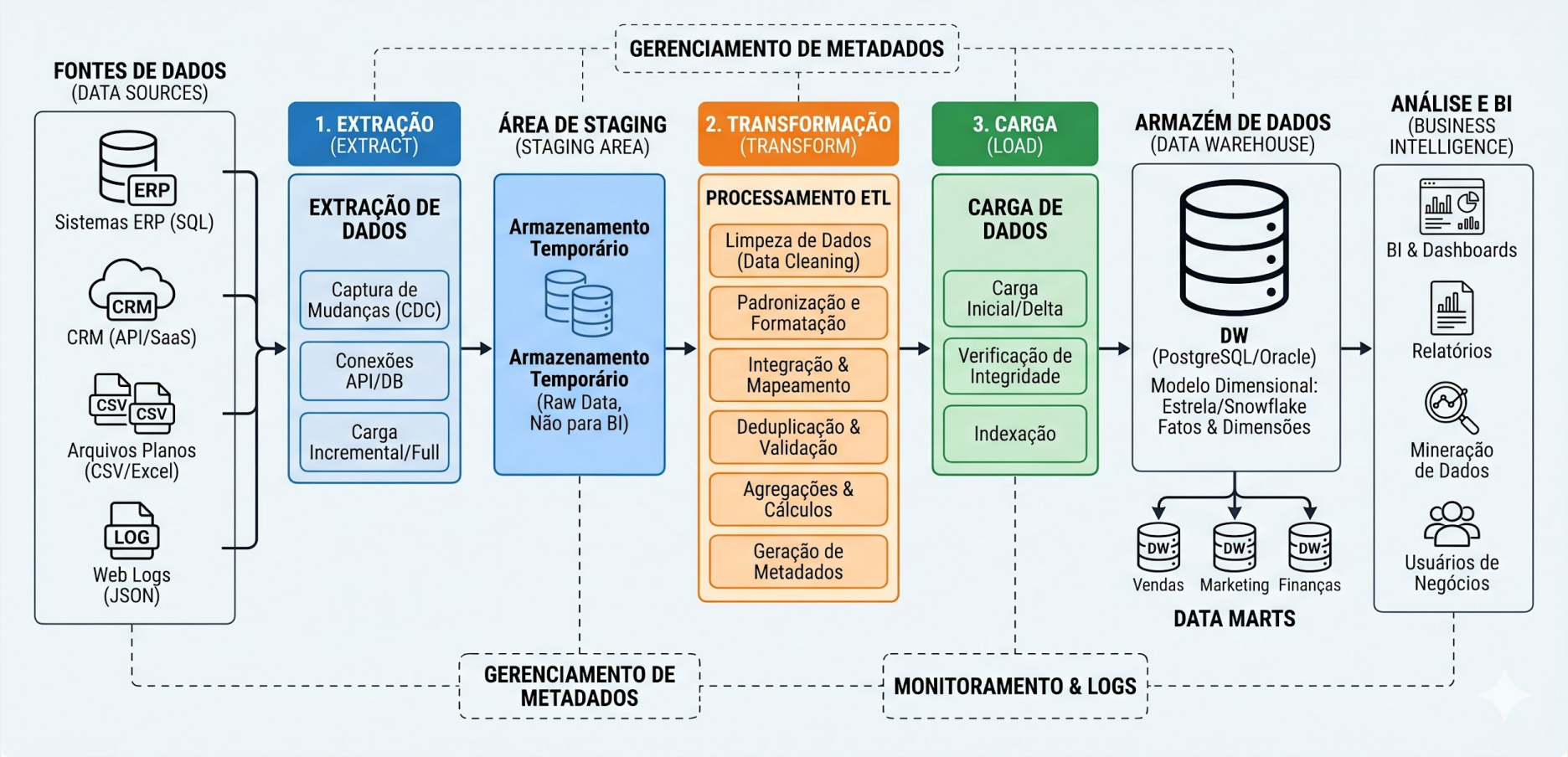
Da Rigidez à Flexibilidade:

A evolução das arquiteturas de dados: de ETLs rígidos a Data Lakes e arquiteturas modernas

01

A Era dos Data Warehouses: ETL e Modelagem Rígida

Arquitetura ETL Tradicional



01 CONTEXTO HISTÓRICO

A Era dos Data Warehouses: ETL e Modelagem Rígida

Pipelines ETL Tradicionais

Os sistemas de **Business Intelligence (BI)** tradicionais dependiam de pipelines de **Extração, Transformação e Carga (ETL)** extremamente rígidos. Esses processos consolidavam informações de diversas fontes em **Data Warehouses centralizados** .

Extração
Coleta de dados

Transformação
Limpeza e padronização

Carga
Armazenamento

Limitações

- Alto tempo de desenvolvimento
- Dificuldade de adaptação a mudanças
- Custo elevado de manutenção
- Dependência de equipes especializadas

Modelagens Complexas

Exigiam **modelagens prévias complexas** , como o modelo estrela ou floco de neve, e esquemas fixos que dificultavam a adaptação a novas necessidades de negócio.

Modelo Estrela
Tabela central de fatos com dimensões ao redor

Modelo Floco de Neve
Dimensões normalizadas em múltiplos níveis

01 CONTEXTO HISTÓRICO

A Revolução do Big Data: Data Lakes

O Surgimento dos Data Lakes

Com o advento do **Big Data** , surgiram os **Data Lakes** , que permitiram o armazenamento de **dados brutos** em seu formato original: estruturados, semi-estruturados e não-estruturados.

Estruturados	Semi-estruturados	Não-estruturados
SQL, CSV	JSON, XML	Texto, Imagens

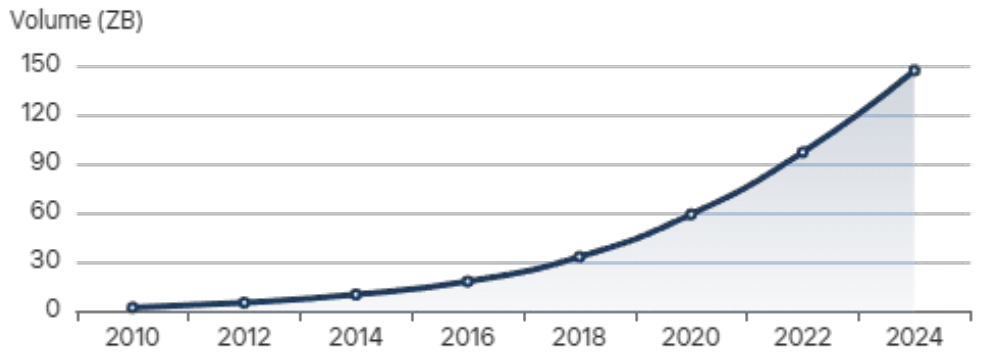
Vantagem: Resolviam problemas de escalabilidade e volume, permitindo armazenar petabytes de dados de forma econômica.

O Problema: Data Swamps

Embora tenham resolvido a escalabilidade, os Data Lakes frequentemente se transformaram em **"pântanos de dados" (Data Swamps)** :

- **Falta de governança:** Dados sem metadados ou documentação
- **Dificuldade de descoberta:** Impossível encontrar informações relevantes
- **Qualidade questionável:** Dados sem validação ou limpeza

Evolução do Volume de Dados



Comparação: Data Warehouse vs Data Lake

Característica	Data Warehouse	Data Lake
Esquema	Pré-definido (schema-on-write)	Flexível (schema-on-read)
Tipos de dados	Estruturados apenas	Todos os tipos
Custo	Alto	Baixo
Usuários	Analistas de negócio	Cientistas de dados

01 CONTEXTO HISTÓRICO

Arquiteturas Modernas: Data Mesh e Data Fabric

Data Mesh

Arquitetura **descentralizada** que trata os dados como **produtos** gerenciados por **domínios de negócio específicos** .

Propriedade Distribuída

Cada domínio (financeiro, marketing, operações) gerencia seus próprios dados

Dados como Produto

Dados são tratados como produtos com SLAs, documentação e governança

Governança Federada

Padrões compartilhados com autonomia dos domínios

Ideal para: Organizações grandes com múltiplas unidades de negócio independentes

Data Fabric

Arquitetura **tecnológica** que utiliza **metadados ativos** para automatizar a integração e o acesso aos dados em ambientes híbridos.

Integração Inteligente

Conecta dados de múltiplas fontes (cloud, on-premise, SaaS) automaticamente

Metadados Ativos

Usa IA para descobrir, classificar e harmonizar dados automaticamente

Governança Centralizada

Políticas consistentes de segurança e compliance em toda a organização

Ideal para: Organizações com dados distribuídos em múltiplos sistemas e clouds

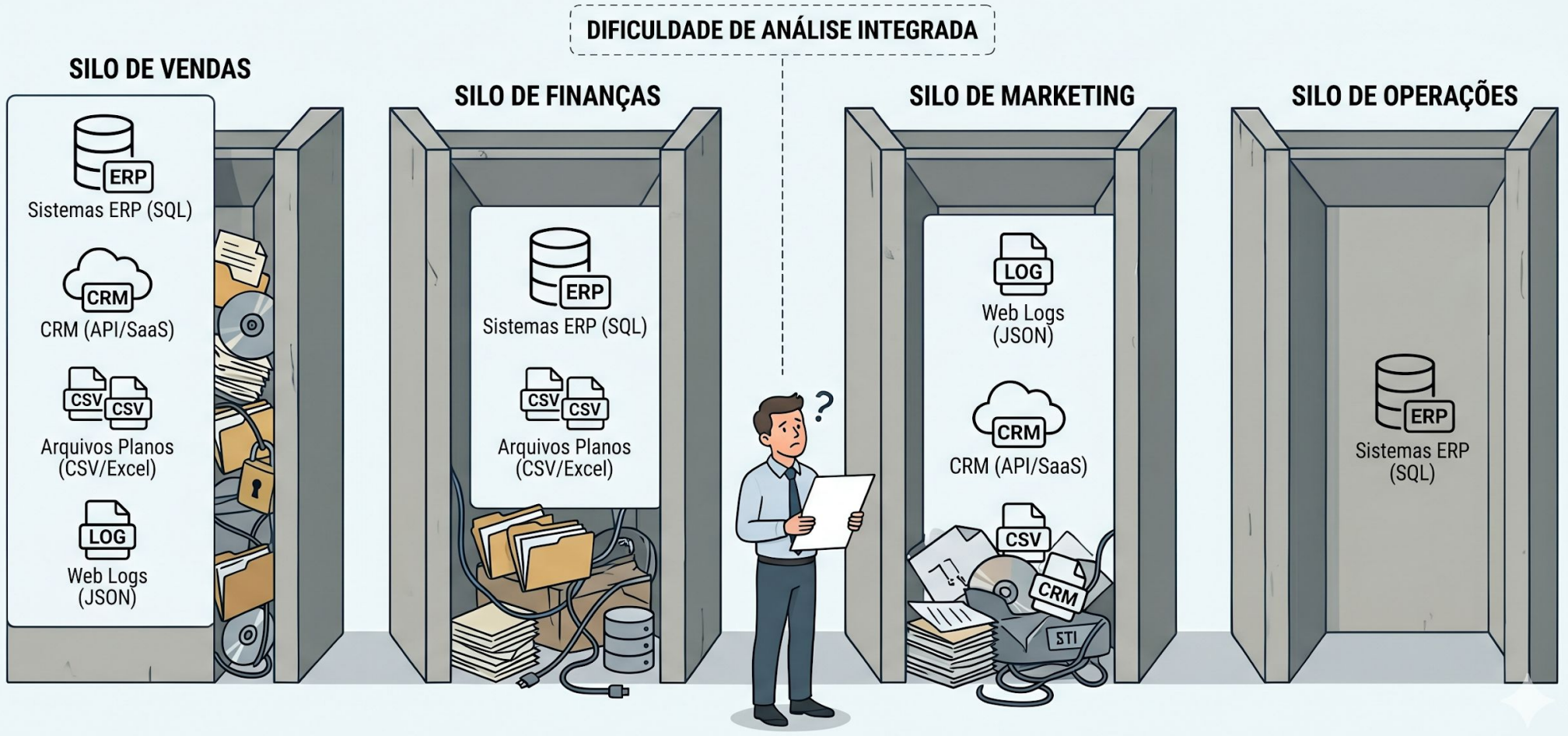
02

O Desafio Atual

Silos de Informação e Heterogeneidade

02 DESAFIO ATUAL

O Cenário Fragmentado das Organizações Modernas



02 DESAFIO ATUAL

O Cenário Fragmentado das Organizações Modernas

Ecossistemas de Dados Heterogêneos

A realidade das organizações modernas é caracterizada por **ecossistemas de dados altamente fragmentados** . As informações estão dispersas em múltiplos sistemas incompatíveis:

Bancos SQL

PostgreSQL, MySQL, Oracle, SQL Server

Sistemas NoSQL

MongoDB, Cassandra, Redis, DynamoDB

APIs SaaS

Salesforce, Zendesk, HubSpot, Workday

Planilhas

Excel, Google Sheets distribuídas

Consequências para o Negócio

- **Perda de produtividade:** Funcionários perdem tempo buscando dados
- **Custos elevados:** Dados de baixa qualidade custam US$ milhões/ano
- **Riscos de segurança:** Organizações com silos podem sofrer com violações/brechas

02 DESAFIO ATUAL

O Gargalo da Integração Ponto-a-Ponto

O Problema da Integração

O principal desafio reside na **dificuldade de extrair conhecimento acionável em tempo real** a partir de silos isolados. A construção de conectores proprietários e integrações ponto-a-ponto é:

Lenta
Desenvolvimento de 3-6 meses por conector

Onerosa
Custo médio de $50K-$200K por integração

Difícil de Manter
Atualizações constantes e dependência técnica

Gargalo para Ciência de Dados e IA

Esse cenário cria um **gargalo significativo** para iniciativas de Ciência de Dados e Inteligência Artificial:

- Cientistas de dados passam 80% do tempo preparando dados
- Projetos de ML são atrasados ou cancelados
- Insights de negócio não chegam em tempo hábil

Complexidade da Integração Ponto-a-Ponto

Custo Acumulado de Integrações

5 sistemas	10 integrações
10 sistemas	45 integrações
20 sistemas	190 integrações

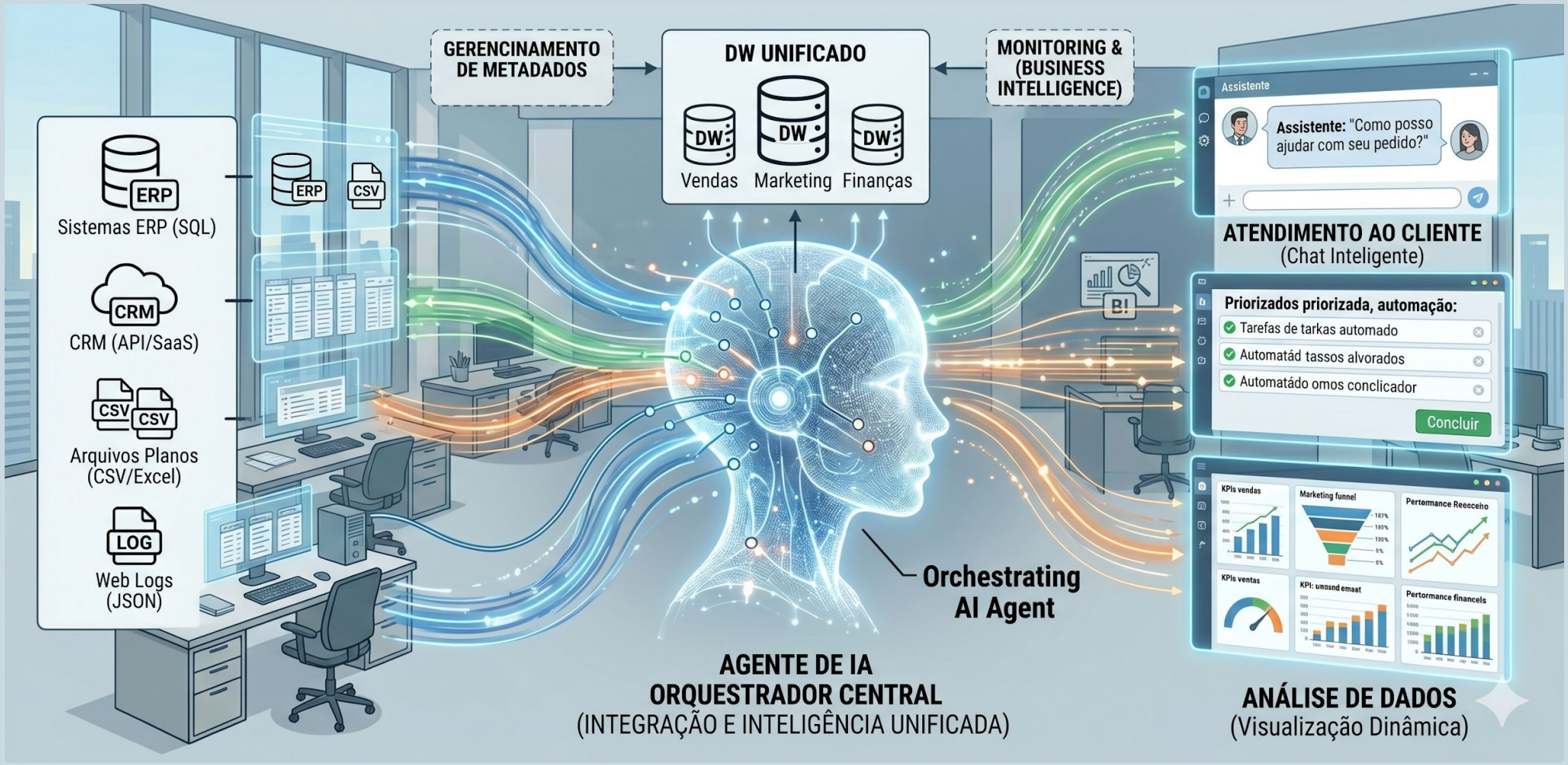
Fórmula: n(n-1)/2 integrações

03

A Ascensão dos Agentes de IA

Como os agentes de IA modernos estão revolucionando a integração de dados

O Que São Agentes de IA Modernos



O Que São Agentes de IA Modernos

Diferença das Abordagens Tradicionais

Diferente das abordagens tradicionais de IA, que focavam em **modelos preditivos** ou **classificadores treinados para tarefas específicas**, os Agentes de IA modernos utilizam **Large Language Models (LLMs)** como motores de raciocínio dinâmico.

IA Tradicional

- Modelos preditivos fixos
- Treinamento supervisionado
- Tarefas específicas
- Respostas determinísticas

Agentes de IA Modernos

- Raciocínio dinâmico com LLMs
- Aprendizado em tempo real
- Múltiplas capacidades
- Respostas contextuais

Um Agente de IA Não é Apenas um Modelo de Linguagem

É um **sistema autônomo** capaz de planejar, raciocinar e executar ações de forma independente para atingir objetivos definidos pelo usuário.

- Compreensão
- Planejamento
- Execução
- Adaptação

Casos de Uso Empresariais

- **Atendimento ao cliente:** Agentes de suporte 24/7
- **Análise de dados:** Consultas em linguagem natural
- **Automação de processos:** Workflows inteligentes
- **Tomada de decisão:** Análise e recomendações

Function Calling: A Ponte entre Linguagem Natural e Dados

O Conceito de Function Calling

Function Calling (ou Tool Calling) é a capacidade dos LLMs modernos de **gerar requisições estruturadas para executar funções externas**, permitindo que agentes interajam com sistemas e bancos de dados.

> **Insight crucial:** O LLM **nunca acessa diretamente** o banco de dados. Ele gera uma **requisição estruturada** que o código executa de forma segura.

1. Definição de Tools

Schemas JSON descrevendo funções disponíveis, parâmetros e quando usá-las

2. Decisão do LLM

O modelo decide qual ferramenta usar e com quais argumentos

3. Execução Segura

Código executa a função e retorna resultado ao LLM

Fluxo de Function Calling

Vantagens da Abordagem

- **Segurança:** Controle total sobre o que o agente pode acessar
- **Flexibilidade:** Mesmas tools para perguntas ilimitadas
- **Confiabilidade:** Estrutura determinística vs. parsing frágil
- **Manutenção:** Schema injetado dinamicamente, sem rebuild

Arquitetura de Agentes: Raciocínio, Planejamento e Memória

Percepção

Capacidade de **interpretar e compreender** a entrada do usuário e o ambiente ao redor.

NLU
Compreensão de linguagem natural

Contexto
Análise de intenção e entidades

Raciocínio

Decompõe problemas complexos em etapas lógicas usando técnicas avançadas.

Chain-of-Thought
Raciocínio sequencial passo a passo

Tree-of-Thought
Exploração de múltiplos caminhos

ReAct
Reason + Act intercalados

Memória

Sistemas de memória que permitem **contexto persistente** e aprendizado contínuo.

Curto Prazo
Contexto da conversa atual

Longo Prazo
Histórico e preferências do usuário

Vetorial
Embeddings para recuperação semântica

Execução de Ferramentas

Invoca funções externas para interagir com o ambiente: **consultas SQL**, **requisições HTTP**, **leitura de arquivos**, **envio de emails**, etc.

- SQL
- APIs
- Arquivos
- Email

Ciclo ReAct

O padrão **ReAct (Reason + Act)** intercala raciocínio com ação para maior adaptabilidade:

1. **Thought:** Raciocina sobre o problema
2. **Action:** Executa uma ação
3. **Observation:** Observa o resultado
4. **Repeat:** Repete até resolver

Agentes como Mediadores Inteligentes

A Ponte entre Mundos

Os agentes atuam como **mediadores inteligentes** entre a **linguagem natural** e as **linguagens de consulta estruturadas** dos sistemas subjacentes.

Benefício principal: Usuários de negócio podem interagir com dados complexos **sem a necessidade de conhecer sintaxe SQL** ou a estrutura de APIs específicas.

Antes

- Dependência de analistas
- Fila de solicitações
- Tempo de espera: dias
- Custo alto por consulta

Com Agentes

- Autoatendimento
- Respostas instantâneas
- Tempo: segundos
- Democratização do acesso

Democratização do Acesso aos Dados

Agentes de IA permitem que **qualquer usuário de negócio** acesse e analise dados complexos:

- ✓ **Gerentes:** Dashboards e relatórios sob demanda
- ✓ **Analistas:** Exploração de dados sem SQL
- ✓ **Executivos:** Insights estratégicos instantâneos

Exemplo Prático

Pergunta do Usuário:

"Qual foi o faturamento por região no último trimestre, comparado com o mesmo período do ano anterior?"

↓

Agente Processa:

1. Compreende intenção e entidades
2. Gera consulta SQL apropriada
3. Executa no banco de dados
4. Formata resposta em linguagem natural

↓

Resposta:

"O faturamento no último trimestre foi de R$ 5,2M, representando um crescimento de 15% comparado ao mesmo período do ano anterior (R$ 4,5M). A região Sudeste liderou com 45% do total..."

ERBD 2026

Fundamentos e Arquitetura do Model Context Protocol

MCP

Prof. Dr. Marlon Marcon
UTFPR - campus Dois vizinhos

UTFPR UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ CÂMPUS DOIS VIZINHOS

SBC Sociedade Brasileira de Computação

Cebd SBC

XXI ERBD 2026 DOIS VIZINHOS - PR de 22 a 24/04

Agenda

01 **Introdução ao MCP**
O que é e por que foi criado

02 **Arquitetura Cliente-Servidor**
Papéis e responsabilidades

03 **Componentes Fundamentais**
Resources, Prompts e Tools

04 **Transporte e Comunicação**
STDIO e SSE

05 **MCP vs RAG Tradicional**
Comparando abordagens

Model Context Protocol

01

Introdução ao Model Context Protocol

O que é MCP e por que ele foi criado

01

PROBLEMA

O Problema da Integração Fragmentada

O Desafio M×N

Cada integração entre LLMs e fontes de dados exigia desenvolvimento **customizado e pontual**, criando um ecossistema fragmentado de conectores específicos.

Consequências

- Redundância de código e esforço
- Dificuldade de manutenção
- Vulnerabilidades de segurança
- Baixa escalabilidade

A Solução: MCP

Uma **camada de interoperabilidade universal** que elimina integrações customizadas através de um protocolo padronizado e aberto.

1	N
Protocolo Único	Servidores

> **Metáfora:** MCP é como o **USB-C para IA** — uma interface universal que conecta qualquer dispositivo.

DEFINIÇÃO

O que é o Model Context Protocol?

Protocolo Aberto

Desenvolvido pela **Anthropic** em novembro de 2024, o MCP é um protocolo aberto e padronizado baseado em **JSON-RPC 2.0** .

Objetivo Principal

Permitir que **LLMs se conectem de forma segura e estruturada** a silos de informação heterogêneos através de uma interface comum.

Como Funciona

1. **Servidores MCP**
   Expoem fontes de dados seguindo o protocolo comum
2. **Clientes MCP**
   Claude Desktop, Cursor, aplicações customizadas
3. **Comunicação**
   Via JSON-RPC 2.0 padronizado

https://github.com/modelcontextprotocol

02

Arquitetura Cliente-Servidor

Entendendo os papéis e responsabilidades

ARQUITETURA

Arquitetura MCP: Cliente e Servidor

Cliente MCP

Aplicação que **hospeda o LLM** e interage diretamente com o usuário.

- Recebe requisições do usuário
- Roteia chamadas para servidores
- Gerencia ciclo de vida da interação

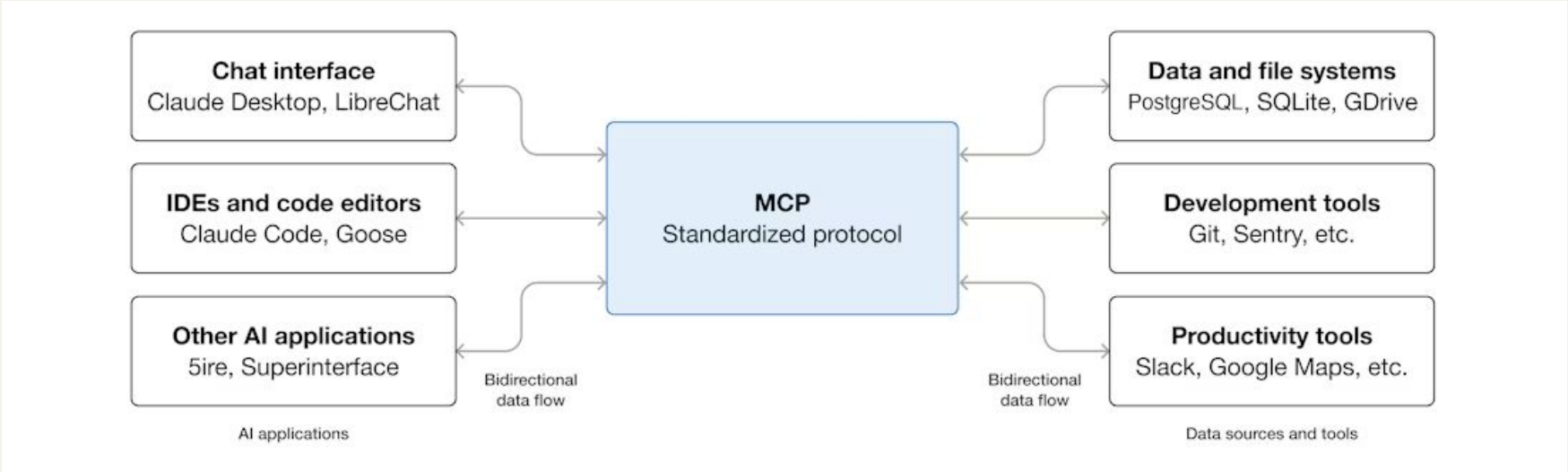
Exemplos: Claude Desktop, Cursor, IDEs

Servidor MCP

Programa **leve e focado** que se conecta a uma fonte de dados específica.

- Conecta a fontes específicas
- Expõe capacidades padronizadas
- Segue protocolo comum

Exemplos: PostgreSQL, GitHub, Slack



CICLO DE VIDA

Ciclo de Vida da Sessão MCP

1 Inicialização

Cliente envia requisição com **versão do protocolo** e **capacidades** suportadas.

Negociação de versão e capacidades

2 Operação

Troca de **requisições, notificações e respostas** conforme capacidades negociadas.

Fluxo bidirecional de mensagens

3 Encerramento

Término **gracioso** da conexão sem mensagens específicas de protocolo.

Fechamento da conexão de transporte

Segurança e Previsibilidade

A negociação de capacidades na inicialização garante que **ambos os lados concordem** com o que pode ser executado, evitando chamadas inesperadas.

- Timeout em requisições
- Tratamento de erros
- Version mismatch

03

Componentes Fundamentais

Resources, Prompts e Tools

03

COMPONENTE 1

Recursos (Resources)

Definição

Dados ou informações **estáticas/dinâmicas** que o servidor disponibiliza para **leitura** , identificados por **URIs customizadas** .

Função Principal

Fornecer **contexto imediato e imutável** (no escopo da requisição) para o LLM, permitindo que o modelo acesse informações relevantes.

Exemplos de Resources

Esquema de Banco de Dados
Estrutura de tabelas e relações

Arquivos de Log
Conteúdo de logs específicos

Documentação de API
Especificações internas

```python
# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"
```

COMPONENTE 2

Prompts

Prompts

Templates **reutilizáveis e parametrizados** que estruturam interações e guiam o LLM para tarefas específicas.

Exemplos:

- "Gerar query SQL para analisar vendas"
- "Resumir logs de erro recentes"

Função: Padronizar fluxos de trabalho

```python
# Add a prompt
@mcp.prompt()
def greet_user(name: str, style: str = "friendly") -> str:
    """Generate a greeting prompt"""
    styles = {
        "friendly": "Please write a warm, friendly greeting",
        "formal": "Please write a formal, professional greeting",
        "casual": "Please write a casual, relaxed greeting",
    }

    return f"{styles.get(style, styles['friendly'])} for someone named {name}."
```

COMPONENTE 3

Ferramentas (Tools)

Ferramentas (Tools)

Funções **executáveis** que permitem ao LLM realizar **ações ativas** no ambiente ou sistemas externos.

Exemplos:

- execute_query - Executar SQL
- fetch_crm_data - Requisição API
- github_search - Buscar no GitHub

Função: Transformar LLM em Agente Ativo

```python
# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b
```

04

04

Transporte e Comunicação

Mecanismos de comunicação entre Cliente e Servidor

TRANSPORTE

Mecanismos de Transporte

STDIO

Standard Input/Output — método mais comum para servidores locais.

- Cliente inicia servidor como subprocesso
- Comunicação via fluxos padrão
- Ideal para mesma máquina

Benefícios: Baixa latência, alta segurança

SSE

Server-Sent Events sobre HTTP — para servidores remotos.

- Comunicação bidirecional HTTP
- Ideal para nuvem e redes corporativas
- Suporte a múltiplos clientes

Benefícios: Escalabilidade, acesso remoto

Escolha do Transporte

STDIO para desenvolvimento local e integrações seguras | **SSE** para produção em nuvem e acesso distribuído

05

MCP vs RAG Tradicional

Comparando abordagens de integração de dados

05

ABORDAGEM 1

RAG Tradicional: Como Funciona

Processo do RAG

1. **Pré-processamento**
   Documentos são processados e divididos em chunks
2. **Vetorização**
   Chunks são convertidos em embeddings vetoriais
3. **Armazenamento**
   Embeddings armazenados em banco de dados vetorial
4. **Busca Semântica**
   Query do usuário é vetorizada e comparada semanticamente
5. **Injeção no Prompt**
   Trechos relevantes são injetados no contexto do LLM

Pontos Fortes

- ✓ Excelente para documentos não estruturados
- ✓ Busca semântica poderosa
- ✓ Base de conhecimento estática

Limitações

- ✗ Dados altamente estruturados
- ✗ Informações em constante mudança
- ✗ Agregações complexas

ABORDAGEM 2

MCP: Consulta Ativa de Dados

Consulta em Tempo Real

O MCP permite que o modelo consulte **dados ativamente** no momento da necessidade usando **Ferramentas (Tools)** .

Exemplo Prático:

Usuário: "Qual o total de vendas do último mês?"

1. LLM identifica necessidade de dado específico
2. Invoca ferramenta
3. Obtém valor **exato e atualizado**

Vantagens do MCP

- Dados em tempo real
- Agregações complexas
- Ações read-write-execute

Comparação RAG vs MCP

Aspecto	RAG	MCP
Tipo de Dados	Estático, não estruturado	Dinâmico, estruturado
Operação	Read-only	Read/Write/Execute
Atualização	Re-embed quando muda	Consulta fonte direta
Melhor Uso	Documentos, FAQs	APIs, Bancos de Dados
Ações	Apenas recuperação	Automação completa

Nota: MCP não substitui RAG — eles se **complementam** em arquiteturas híbridas

DECISÃO

Quando Usar Cada Abordagem

Use RAG

Dados Estáticos
Documentos que não mudam frequentemente

Não Estruturados
PDFs, textos, artigos, políticas

Read-Only
Apenas recuperação de informação

Busca Semântica
Similaridade de significado

Ex: FAQs, Documentação

Use MCP

Dados Dinâmicos
Informações em constante mudança

Estruturados
Bancos SQL, APIs REST

Read-Write-Execute
Ações e automações

Tempo Real
Dados atualizados no momento

Ex: CRM, ERP, APIs

Use Ambos

Arquitetura Híbrida
Quando conhecimento informa ação

Padrão Sequencial

1. RAG recupera política
2. MCP executa ação
3. Resposta combinada

Exemplo: Reembolso
RAG verifica política → MCP processa reembolso

Mais comum em produção

Obrigado!

Prof. Dr. Marlon Marcon
marlonmarcon@utfpr.edu.br